

✓ Projeto de Machine Learning – Eficiência Energética de Edificações

Contexto

O consumo energético em edificações é um fator essencial para eficiência operacional e sustentabilidade. Neste trabalho, busca-se analisar e prever o consumo energético com base em características estruturais.

Inspirado em conceitos de Controle Estatístico de Processo (CEP), este estudo também analisa a variabilidade dos dados e o comportamento do processo.

Formulação do Problema

Trata-se de um problema de aprendizado supervisionado, no qual deseja-se aprender uma função $f(X) \rightarrow Y$, onde:

- X: características das edificações
- Y: consumo energético (Heating Load)

O objetivo é construir um modelo capaz de generalizar para novos dados.

Dataset: https://archive.ics.uci.edu/ml/machine-learning-databases/00242/ENB2012_data.xlsx

Hipóteses

- Características estruturais influenciam o consumo energético
- Existe variabilidade significativa no processo

```
1 # IMPORTAÇÃO E DADOS
2
3 import pandas as pd
4 import numpy as np
5
6 url = "https://archive.ics.uci.edu/ml/machine-learning-databases/00242/ENB2012_data.xlsx"
7 df = pd.read_excel(url)
8
9 df.head()
```

	X1	X2	X3	X4	X5	X6	X7	X8	Y1	Y2
0	0.98	514.5	294.0	110.25	7.0	2	0.0	0	15.55	21.33
1	0.98	514.5	294.0	110.25	7.0	3	0.0	0	15.55	21.33
2	0.98	514.5	294.0	110.25	7.0	4	0.0	0	15.55	21.33
3	0.98	514.5	294.0	110.25	7.0	5	0.0	0	15.55	21.33
4	0.90	563.5	318.5	122.50	7.0	2	0.0	0	20.84	28.28

```
1 # EXPLORAÇÃO DOS DADOS
2
3 df.info()
4 df.describe()
5 df.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 10 columns):
#   Column  Non-Null Count  Dtype
---  -
0    X1      768 non-null     float64
1    X2      768 non-null     float64
2    X3      768 non-null     float64
3    X4      768 non-null     float64
4    X5      768 non-null     float64
5    X6      768 non-null     int64
6    X7      768 non-null     float64
7    X8      768 non-null     int64
8    Y1      768 non-null     float64
9    Y2      768 non-null     float64
dtypes: float64(8), int64(2)
memory usage: 60.1 KB
```

```

0
X1  0
X2  0
X3  0
X4  0
X5  0
X6  0
X7  0
X8  0
Y1  0
Y2  0
```

```
dtype: int64
```

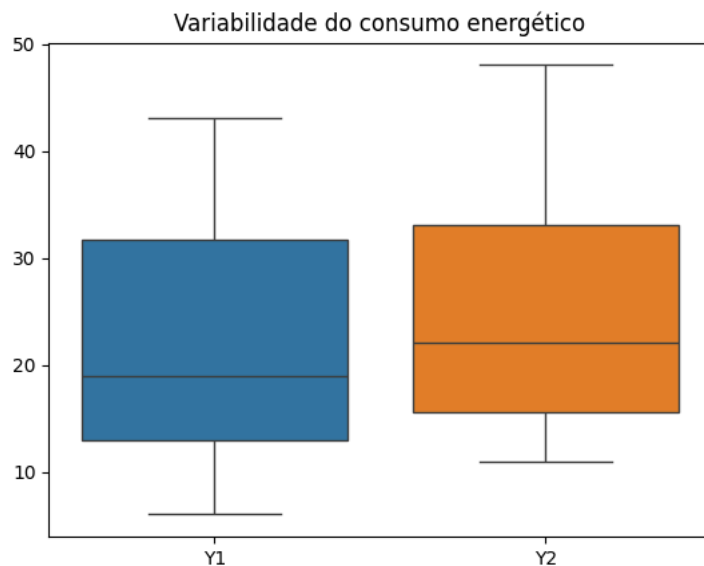
O dataset contém variáveis relacionadas às características físicas das edificações e duas variáveis alvo:

- Y1: Heating Load
- Y2: Cooling Load

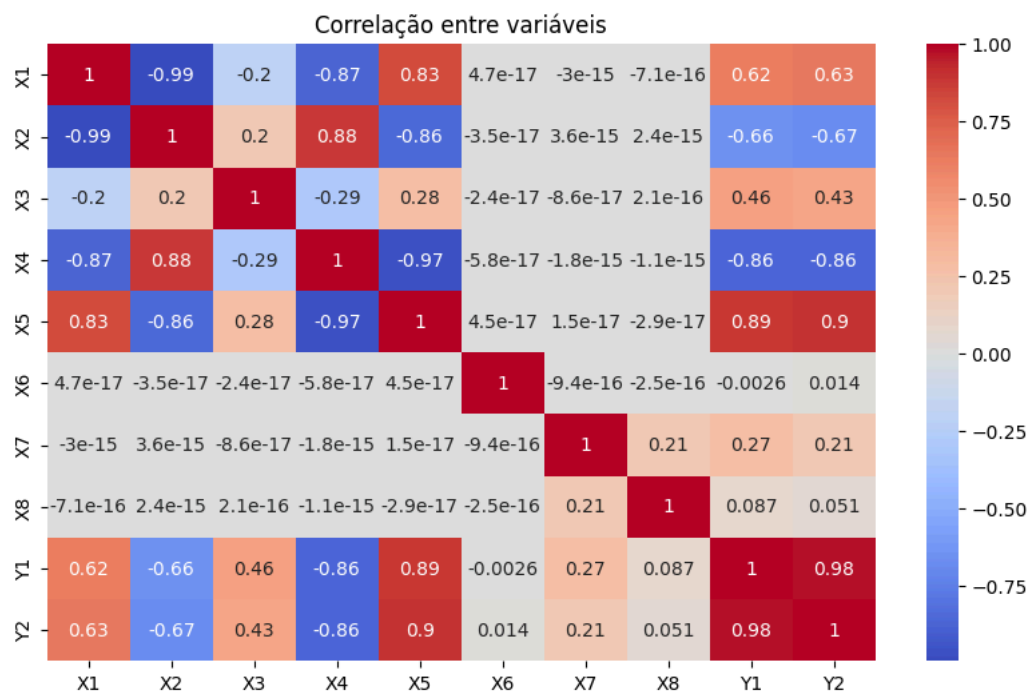
```
1 # PRÉ-PROCESSAMENTO
2
3 df = df.dropna()
4
5 X = df.drop(['Y1','Y2'], axis=1)
6 y = df['Y1']
```

```
1 # TREINO E TESTE
2
3 from sklearn.model_selection import train_test_split
4
5 X_train, X_test, y_train, y_test = train_test_split(
6     X, y, test_size=0.2, random_state=42
7 )
```

```
1 # ANÁLISE EXPLORATÓRIA
2
3 import seaborn as sns
4 import matplotlib.pyplot as plt
5
6 sns.boxplot(data=df[['Y1','Y2']])
7 plt.title("Variabilidade do consumo energético")
8 plt.show()
```



```
1 plt.figure(figsize=(10,6))
2 sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
3 plt.title("Correlação entre variáveis")
4 plt.show()
```



Observa-se variabilidade significativa nos dados, indicando diferenças no comportamento energético das edificações, o que é compatível com sistemas reais.

```
1 # MODELAGEM
2
3 from sklearn.linear_model import LinearRegression
4 from sklearn.ensemble import RandomForestRegressor
5 from sklearn.tree import DecisionTreeRegressor
6 from sklearn.neighbors import KNeighborsRegressor
7
8 model1 = LinearRegression()
9 model2 = RandomForestRegressor()
10 model3 = DecisionTreeRegressor()
11 model4 = KNeighborsRegressor()
12
13 model1.fit(X_train, y_train)
14 model2.fit(X_train, y_train)
15 model3.fit(X_train, y_train)
16 model4.fit(X_train, y_train)
```

```
-----
NameError                                Traceback (most recent call last)
/tmp/ipykernel_22490/3663540191.py in <cell line: 0>()
    11 model4 = KNeighborsRegressor()
    12
--> 13 model1.fit(X_train, y_train)
    14 model2.fit(X_train, y_train)
    15 model3.fit(X_train, y_train)

NameError: name 'X_train' is not defined
```

```
1 # AVALIAÇÃO
2
3 from sklearn.metrics import mean_squared_error, r2_score
4
5 models = {
6     "Linear Regression": model1,
7     "Random Forest": model2,
8     "Decision Tree": model3,
9     "KNN": model4
10 }
11
12 for name, model in models.items():
13     pred = model.predict(X_test)
14     print(f"\n{name}")
15     print("MSE:", mean_squared_error(y_test, pred))
16     print("R2:", r2_score(y_test, pred))
```

```
-----
NameError                                Traceback (most recent call last)
/tmp/ipykernel_22490/1090738706.py in <cell line: 0>()
    11
    12 for name, model in models.items():
--> 13     pred = model.predict(X_test)
    14     print(f"\n{name}")
    15     print("MSE:", mean_squared_error(y_test, pred))

NameError: name 'X_test' is not defined
```

```
1 # VALIDAÇÃO CRUZADA
2
3 from sklearn.model_selection import cross_val_score
4
5 scores = cross_val_score(model2, X, y, cv=5)
6
7 print("Scores:", scores)
8 print("Média:", scores.mean())
9 print("Desvio:", scores.std())
```

```
Scores: [0.84169017 0.99589631 0.9968852  0.99659889 0.99411375]
Média: 0.9650368636577349
Desvio: 0.0616808806081106
```

A validação cruzada fornece uma estimativa mais robusta da capacidade de generalização do modelo.

```
1 # OTIMIZAÇÃO
2
3 from sklearn.model_selection import GridSearchCV
4
5 params = {'n_estimators':[50,100,150]}
6
7 grid = GridSearchCV(RandomForestRegressor(), params)
8 grid.fit(X_train, y_train)
9
10 print("Melhores parâmetros:", grid.best_params_)
```

```
Melhores parâmetros: {'n_estimators': 150}
```

```
1 # IMPORTÂNCIA DAS VARIÁVEIS
2
3 import pandas as pd
4
5 importances = model2.feature_importances_
6 features = X.columns
7
8 df_imp = pd.DataFrame({'Feature':features,'Importance':importances})
9 df_imp.sort_values(by='Importance', ascending=False)
```

	Feature	Importance
0	X1	0.423161
1	X2	0.162501
3	X4	0.159395
4	X5	0.118474
6	X7	0.078823
2	X3	0.043664
7	X8	0.013222
5	X6	0.000759